

p0332R2

Relaxed Incomplete Multidimensional Array

Type Declaration

Published Proposal, 10 February 2018

This version:

https://kokkos.github.io/array_ref/proposals/P0332.html

Authors:

[H. Carter Edwards](#)

[Daniel Sunderland](#)

[Christian Trott](#)

[David Hollman](#)

[Mauro Bianco](#)

[Athanasios Iliopoulos](#)

[John Michopoulos](#)

Audience:

EWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Significantly improve usability of the `mdspan` multidimensional array library by providing users with a concise and intuitive syntax for specifying static and dynamic dimensions.

Table of Contents

1	Revision History
1.1	[N4356] : Original proposal, 2015-Lenexa
1.2	[P0332r0]
1.3	[P0332r1]
1.4	P0332r2
2	Motivation
2.1	Array type for <code>mdspan</code>
2.2	Why Not
2.3	Explicit and Implicit Extents
2.4	Usability by Mathematicians, Scientists, and Engineers
2.5	Clarifying difference between array type and array object declarations
3	Proposal
3.1	Clarification of incomplete types
3.2	Clarification and relaxation of array type
3.3	<code>type_traits</code> interaction
4	Precedence and Feasibility
5	Holistic View
5.1	2015-Lenexa EWG discussion on N4356
5.2	Analysis with respect to N4700 working draft

- Type Deduction Non-Issue
- 5.3
- 5.4 Array Types of Zero Length
- 5.5 Other Potential Applications

References

Informative References

§ 1. Revision History

§ 1.1. [\[N4356\]](#) : Original proposal, 2015-Lenexa

§ 1.2. [\[P0332r0\]](#)

- Relaxed array declaration extracted from [\[P0009r0\]](#)

§ 1.3. [\[P0332r1\]](#)

- Align with updates to [\[P0009r3\]](#)
- Add examples of precedents
- Reviews at 2017-Albuquerque
 - LEWG: moves [\[P0009r0\]](#) to LWG
 - EWG feedback: [\[P0332r1\]](#) fails to address concerns raised at 2015-Lenexa regarding N4356

§ 1.4. [P0332r2](#)

- Fully address scope of 2015-Lenexa EWG feedback on [\[N4356\]](#)
- Introduce clarification for loosely defined *incomplete type*

§ 2. Motivation

§ 2.1. Array type for `mdspan`

The dimensions of multidimensional array reference `mdspan` ([\[P0331r0\]](#) and [\[P0009r3\]](#)) are declared with a syntactically verbose extents property argument. We propose a minor, non-breaking relaxation of the array type declaration in `[decl.array]` to allow a concise and intuitive syntax for multidimensional declarations.

```
template< typename DataType , typename Properties... >
struct mdspan ;

// Three dimensional tensor type declaration with
// verbose syntax and left-to-right increasing stride.

using tensor = std::mdspan<double, std::extents<std::dynamic_extent, std::dynamic_extent, std::dynamic_extent>, std::layout_left> ;

// Three dimensional tensor type declaration with concise syntax
// and left-to-right increasing stride.

using tensor = mdspan<double[][], std::layout_left> ;
```

The motivating `mdspan` multidimensional array library proposal ([\[P0009r3\]](#)) was moved by LEWG to LWG in 2017-Albuquerque. Throughout LEWG discussions of the multidimensional array proposal, the

consensus in LEWG has been that the usability of the `mdspan` library would be significantly improved with the relaxed array type syntax of this proposal.

§ 2.2. Why Not

```
mdspan<double, std::dynamic_extent, std::dynamic_extent, std::dynamic_extent, std::LayoutLeft>
```

- Does not allow `layout_left` or other properties to be types.
- Still requires `dynamic_extent` magic number.

§ 2.3. Explicit and Implicit Extents

The `mdspan` supports declarations with both explicit (compile time) and implicit (runtime) extents. Explicit extents enables optimization of array indexing computations but not all extents can be explicitly known. For this reason the Eigen library defines two dimensional array (matrix) types where either column or row extents may be explicit. For higher ranks the mix of explicit and implicit extents becomes syntactically unwieldy; which may be why n-dimensional arrays in the TensorFlow library does not provide this capability.

§ 2.4. Usability by Mathematicians, Scientists, and Engineers

Multidimensional arrays are a fundamental mathematical building block in science and engineering. As such the FORTRAN programming language created over five decades ago by this community for this community includes multidimensional arrays as a fundamental component of the language. Decades of investment in FORTRAN compiler technology enabled high levels of optimization for mathematical kernels using nested loops to operate on multidimensional arrays.

When the C and C++ languages were created, innovators in the computational mathematical, scientific, and engineering disciplines realized the benefits of abstraction-enabling features in these languages. As such numerous development teams switched from FORTRAN to C++ for their applications. However, because C++ did not support multidimensional arrays as usable and optimizable as those in FORTRAN the state-of-the-practice became for C++ applications to use C++ for higher level abstractions and call FORTRAN routines for lower level, performance critical kernels.

Changes in computational hardware, such as the re-introduction of wide vector units in CPUs and mainstreaming of GPUs for computing, created a performance-portability problem; different architectures require different data layouts to achieve performance. The proposed multidimensional array library ([P0009r3]) provides the mechanism to solve this problem; `mdspan` with polymorphic data-layout. The `mdspan` library is an opportunity for C++ to solve the data-layout problem and reclaim performance parity with FORTRAN.

The proposed `mdspan` library provides the necessary features for performant and portable multidimensional arrays on diverse modern computational hardware architectures. However `mdspan` has an usability Achilles heel: the current `mdspan` syntax for declaring a multidimensional array type is extremely verbose and unpalatable to the computational mathematicians, scientists, and engineers who are the primary users of multidimensional array data structures. The minor, non-breaking change for relaxed multidimensional array type declarations in this proposal solves the usability problem by providing `mdspan` with a concise, intuitive, and highly usable syntax.

§ 2.5. Clarifying difference between array type and array object declarations

An array object has the unusual ability to change array type, as illustrated in the following example.

```
extern int x[];
using T_incomplete = decltype(x);
int x[42];
using T_complete = decltype(x)
static_assert( ! is_same_v<T_incomplete,T_complete> );
```

Specifications ([N4700]) to support this unusual ability conflate the rules for array type and array object declarations. In this proposal we systematically reviewed complete array type, incomplete array type, and array object declaration rules in ([N4700]) and propose revisions to disambiguate these rules.

§ 3. Proposal

This proposal has two themes: (1) clarification of incomplete types, array type declarations, and array object declarations and (2) relaxation of extent expressions for incomplete array types but not array object declarations.

§ 3.1. Clarification of incomplete types

N4700: [types.basic] 6.9 Types, p5-p6

A class that has been declared but not defined, an enumeration type in certain contexts (10.2), or an array of unknown bound or of incomplete element type, is an incompletely-defined object type. Incompletely-defined object types and cv void are incomplete types (6.9.1). Objects shall not be defined to have an incomplete type.

A class type (such as "class X") might be incomplete at one point in a translation unit and complete later on; the type "class X" is the same type at both points. The declared type of an array object might be an array of incomplete class type and therefore incomplete; if the class type is completed later on in the translation unit, the array type becomes complete; the array type at those two points is the same type. The declared type of an array object might be an array of unknown bound and therefore be incomplete at one point in a translation unit and complete later on; the array types at those two points ("array of unknown bound of T" and "array of N T") are different types. The type of a pointer to array of unknown bound, or of a type defined by a typedef declaration to be an array of unknown bound, cannot be completed.

Proposed: [types.basic] 6.9 Types, p5-p6

An *incomplete class type* is a class that has been declared but not defined. A class type (such as "`class X`") might be incomplete at one point in a translation unit and completed later in the translation unit; the type "`class X`" is the same type at both points.

When the element type `T` of an "array of `N T`" (such as "`T[N]`") is an incomplete class type the array type is incomplete; if the class type is later completed in the translation unit the array type becomes complete and the array type at those two points is the same type.

When the declared type of an array object is of unknown bound (such as "`T obj[]`") at one point in a translation unit and is later completed in the translation unit (such as "`T obj[N]`") the array types at those two points ("`T[]`" and "`T[N]`") are different types.

The type of a pointer to array of unknown bound, or of a type defined to be an array of unknown bound, cannot be completed.

Restrictions on the element type `T` for an "array of `N T`" and "array of unknown bound of `T`" are specified in 11.3.4, Arrays.

An *incompletely-defined object type* is

- an incomplete class type,
- an enumeration type in certain contexts (10.2),
- an array of `N T` (11.3.4) where `T` is an incomplete class type, or
- an array of unknown bound of `T` (11.3.4) where `T` is an object type.

An *incomplete type that cannot be completed* is

- `cv void` (6.9.1),
- an array of unknown bound of `T`,
- a pointer to an incomplete type that cannot be completed other than `cv void`, or
- an array of `T` where element type `T` is an incomplete type that cannot be completed.

An *incomplete type* is

- an incompletely-defined object type or
- an incomplete type that cannot be completed.

Add to example:

```
typedef int UNKA[];           // UNKA is an incomplete type
typedef UNKA UNKAA[N];       // UNKAA is an incomplete type that cannot be completed
UNKA arrn[N];                // ill-formed, UNKA cannot be completed
UNKA* arrp;                   // ill-formed, UNKA* cannot be completed
```

N4700: [types.basic] 6.9 Types, p8

An object type is a (possibly cv-qualified) type that is not a function type, not a reference type, and not cv void.

Proposed: [types.basic] 6.9 Types, p8

An *object type* is a (possibly cv-qualified) type that is not a function type, not a reference type, and not an incomplete type that cannot be completed.

§ 3.2. Clarification and relaxation of array type

N4700: [dcl.array] 11.3.4 Arrays, p1

In a declaration `T D` where `D` has the form

```
D1 [ constant-expression_opt ]  
attribute-specifier-seq_opt
```

and the type of the identifier in the declaration `T D1` is "derived-declarator-type-list `T`", then the type of the identifier of `D` is an array type; if the type of the identifier of `D` contains the auto type-specifier, the program is ill-formed. `T` is called the array element type; this type shall not be a reference type, cv void, a function type or an abstract class type. If the constant-expression (8.20) is present, it shall be a converted constant expression of type `std::size_t` and its value shall be greater than zero. The constant expression specifies the bound of (number of elements in) the array. If the value of the constant expression is `N`, the array has `N` elements numbered 0 to `N-1`, and the type of the identifier of `D` is "derived-declarator-type-list array of `N T`". An object of array type contains a contiguously allocated non-empty set of `N` subobjects of type `T`. Except as noted below, if the constant expression is omitted, the type of the identifier of `D` is "derived-declarator-type-list array of unknown bound of `T`", an incomplete object type. The type "derived-declarator-type-list array of `N T`" is a different type from the type "derived-declarator-type-list array of unknown bound of `T`", see 6.9. Any type of the form "cv-qualifier-seq array of `N T`" is adjusted to "array of `N cv-qualifier-seq T`", and similarly for "array of unknown bound of `T`". The optional attribute-specifier-seq appertains to the array.

Proposed: [dcl.array] 11.3.4 Arrays, p1; Clarify the difference between an *array type declaration* and *array object declaration*.

In an *array type declaration*

```
T [ constant-expression_opt ]  
typedef T D1 [ constant-expression_opt ]  
using D1 = T [ constant-expression_opt ]
```

`T` is the array *element type*; this type shall not be a reference type, a function type, an abstract class, or cv void. If the *constant-expression* (8.20) is present, it is a converted constant expression of type `std::size_t`. If the value of the constant expression is `N`, the array type is "array of `N T`". The constant expression specifies the *bound* of (number of elements in) objects of the array type. If the constant expression is omitted the type is an "array of unknown bound of `T`" and is an incomplete type (6.9). The type "array of `N T`" is a different type from the type "array of unknown bound of `T`" (6.9). Any type of the form "cv-qualifier-seq array of `N T`" is adjusted to "array of `N cv-qualifier-seq T`", similarly for "array of unknown bound of `T`". If the element type is an incomplete type that cannot be completed or an *array of unknown bound of U*, where `U` is any type, then the array type is an incomplete type that cannot be completed.

In an *array object declaration* `T D` where `D` has the form

```
D1 [ constant-expression_opt ]  
attribute-specifier-seq_opt
```

and the type of the identifier in the declaration `T D1` is "derived-declarator-type-list `T`", then the type of the identifier `D` is an array type `T [ constant-expression_opt ]`. If the type of the identifier of `D` contains the auto type-specifier, the program is ill-formed. `T` is called the array *element type*; this type shall not be a reference type, a function type, an abstract class, or an incomplete type that cannot be completed. Except as noted below, the constant expression shall not be omitted. The optional *attribute-specifier-seq* appertains to the array object. If the value of the constant expression is `N`, the array has `N` elements numbered 0 to `N-1`, and the type of the identifier of `D` is "derived-declarator-type-list array of `N T`". An array object contains a contiguous non-empty set of `N` subobjects of type `T` numbered 0 to `N-1`.

N4700: [dcl.array] 11.3.4 Arrays, p2

An array can be constructed from one of the fundamental types (except void), from a pointer, from a pointer to member, from a class, from an enumeration type, or from another array.

Proposed: [dcl.array] 11.3.4 Arrays, p2

An array type can be declared with element type of one of the fundamental types (except void), a pointer, a pointer to member, a class, an enumeration type, or another array type.

An array object can be declared with any array type except one that is an incomplete type that cannot be completed.

N4700: [dcl.array] 11.3.4 Arrays, p3

When several "array of" specifications are adjacent, a multidimensional array type is created; only the first of the constant expressions that specify the bounds of the arrays may be omitted. In addition to declarations in which an incomplete object type is allowed, an array bound may be omitted in some cases in the declaration of a function parameter (11.3.5). An array bound may also be omitted when the declarator is followed by an initializer (11.6) or when a declarator for a static data member is followed by a brace-or-equal-initializer (12.2). In both cases the bound is calculated from the number of initial elements (say, N) supplied (11.6.1), and the type of the identifier of D is "array of N T". Furthermore, if there is a preceding declaration of the entity in the same scope in which the bound was specified, an omitted array bound is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class.

Proposed: [dcl.array] 11.3.4 Arrays, p3

When several "array of" specifications are adjacent, a multidimensional array type is created. In declarations in which an *incomplete type* is allowed any of the constant expressions that specify the bounds of the arrays may be omitted; if any of the constant expressions are omitted the type is an incomplete type that cannot be completed. The first of the constant expressions that specify the bounds of the arrays may be omitted

- in some cases in the declaration of a function parameter (11.3.5),
- when the declarator is followed by an initializer (11.6),
- when a declarator for a static data member is followed by a brace-or-equal-initializer (12.2), or
- if there is a preceding declaration of the entity in the same scope in which the bound was specified.

In the initializer cases the bound is calculated from the number of initial elements (say, N) supplied (11.6.1), and the type of the identifier of D is "array of N T". In the preceding declaration case an omitted array bound is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class.

§ 3.3. type_traits interaction

```
using S = double[10][20][] ;
rank_v<S> == 3
extent_v<S,0> == 10
extent_v<S,1> == 20
extent_v<S,2> == 0

remove_extent_t<S> // is an incomplete type
is_same_v< remove_extent_t<S> , double[20][] >

remove_extent_t< remove_extent_t<S> > // is an incomplete type
is_same_v< remove_extent_t< remove_extent_t<S> > , double[] >

decay_t<S> // is an incomplete type
is_same_v< decay_t<S> , double(*)[20][] >
```

§ 4. Precedence and Feasibility

An incomplete array type `T[]` to concisely indicate an array of runtime length is used by `std::unique_ptr<T[]>` (23.11.1.3), `std::shared_ptr<T>` where `T` is `U[]` (23.11.2.2), and [\[P0674r1\]](#) `make_shared<T[] [N1] [N2]>`.

This minor language specification change has been implemented with a simple patch to Clang.

§ 5. Holistic View

§ 5.1. 2015-Lenexa EWG discussion on N4356

"Stepping back for a second, I think this is a small change but there are a whole bunch of ways of constructing types and we disallow many because they would give uninhabited types. But then look at `std::result_of`, after this change you can use `std::result_of` on a whole bunch of types, but not on a function type (ironically). I think there may be some sense in this, I'd like to see some more holistic view of this, I don't want to see pointers or references to these, or functions declared with these things as arguments."

§ 5.2. Analysis with respect to N4700 working draft

Let `S` be an incomplete multidimensional array type greater than rank 1 from which an extent other than the leading extent is omitted.

N4700 [basic.link] 6.5 Program and linkage, p10

After all adjustments of types (during which typedefs (10.1.3) are replaced by their definitions), the types specified by all declarations referring to a given variable or function shall be identical, except that declarations for an array object can specify array types that differ by the presence or absence of a major array bound (11.3.4). A violation of this rule on type identity does not require a diagnostic.

Array object declarations restricted to absence of only the leading array bound.

N4700 [types.basic] 6.9 Types, p5

incompletely-defined object type

A class that has been declared but not defined, an enumeration type in certain contexts (10.2), or an array of unknown bound or of incomplete element type, is an incompletely-defined object type. Incompletely-defined object types and cv void are incomplete types (6.9.1). Objects shall not be defined to have an incomplete type.

[footnote] The size and layout of an instance of an incompletely-defined object type is unknown.

An array of unknown bound is an incomplete type, so `S` can never be used to declare an object.

N4700 [types.basic] 6.9 Types, p6

The declared type of an array object might be an array of unknown bound and therefore be incomplete at one point in a translation unit and complete later on; the array types at those two points ("array of unknown bound of `T`" and "array of `N T`") are different types. The type of a pointer to array of unknown bound, or of a type defined by a typedef declaration to be an array of unknown bound, cannot be completed.

The type of a pointer to `S` is an incomplete type that cannot be completed and therefore can never be used to declare an object.

N4700 [basic.fundamental] 6.9.1 Fundamental types, p9

A type cv void is an incomplete type that cannot be completed; such a type has an empty set of values.

An incomplete multidimensional array type in which an extent other than the first extent is omitted cannot be completed.

N4700 [basic.type.qualifier] 6.9.3 CV-qualifiers, p1

Each type which is a cv-unqualified complete or incomplete object type or is void (6.9)...

CV-qualifiers apply to complete or incomplete types.

N4700 [conf.array] 7.2 Array-to-pointer conversion

An lvalue or rvalue of type "array of N T" or "array of unknown bound of T" can be converted to a prvalue of type "pointer to T". The temporary materialization conversion (7.4) is applied. The result is a pointer to the first element of the array.

As is, T cannot be an *incomplete type that cannot be completed*, such as `void`. This proposal does not change this fact.

N4700 [conv.rval] 7.4 Temporary materialization conversion [conv.rval]

A prvalue of type T can be converted to an xvalue of type T. This conversion initializes a temporary object (15.2) of type T from the prvalue by evaluating the prvalue with the temporary object as its result object, and produces an xvalue denoting the temporary object. T shall be a complete type.

The decay of `int[][M][]` is `int(*)[M][]` which is an incomplete type that cannot be completed, and objects cannot be declared of this type. Therefore converting `int[][M][]` to a pointer is an error.

N4700 [expr.call] 8.2.2 Function call, p4

When a function is called, the parameters that have object type shall have completely-defined object type. [Note: this still allows a parameter to be a pointer or reference to an incomplete class type. However, it prevents a passed-by-value parameter to have an incomplete class type. ---end note]

A parameter is not allowed to be a pointer or reference to an incomplete array type.

N4700 [expr.throw] 8.17 Throwing an exception, p2

Evaluating a throw-expression with an operand throws an exception (18.1); the type of the exception object is determined by removing any top-level cv-qualifiers from the static type of the operand and adjusting the type from "array of T" or function type T to "pointer to T".

N4700 [dcl.array] 11.3.4 Arrays, p2

An array can be constructed from one of the fundamental types (except void), from a pointer, from a pointer to member, from a class, from an enumeration type, or from another array.

"Another array" may be an array of unknown bound.

N4700 [dlc.fct] 11.3.5 Functions, p5

After determining the type of each parameter, any parameter of type "array of T" or of function type T is adjusted to be "pointer to T".

Constrain such that T is a complete type or an incomplete class type.

N4700 [dcl.stc] Storage class specifiers, p7

The name of a declared but undefined class can be used in an extern declaration. Such a declaration can only be used in ways that do not require a complete class type.

Incomplete array types cannot be used as the return type of a function.

§ 5.3. Type Deduction Non-Issue

Incomplete array types observe normal type deduction and pointer decay rules.

```
template <typename T>
void f( mdspace<T[3][5]> ); // A

template <typename T>
void f( mdspace<T[1][3][5]> ); // B

template <typename T>
void f( mdspace<T[1][][5]> ); // C

template <typename T, std::size_t M, std::size_t N>
void f( mdspace<T[N][M][]> ) // D

template <typename T, std::size_t M, std::size_t N>
void f( mdspace<T[][N][M]> ); // E

template <typename T>
void f( T[][3][5] ); // F
    // adjusted to pointer T(*)[3][5]

template <typename T>
void f( T[][][5] ); // G
    // adjusted to pointer T(*)[][5] which is
    // invalid due to T[][5] incomplete array type

template <typename T, std::size_t M >
void f( T[][M][] ); // H
    // adjusted to pointer T(*)[M][] which is
    // invalid due to T[][5] incomplete array type

template <typename T, std::size_t M, std::size_t N>
void f( T[][N][M] ); // I
    // adjusted to pointer T(*)[M][N]

int foo( mdspace<int[1][3][5]> x )
{
    f(x); // no ambiguity
    // COULD match A with T == int[1]
    // DOES match B with T == int ; more specialized
    // NOT match D because [3] != []
    // NOT match E because [5] != []
    // NOT match F because [1] != []
}

int foo( int y[][3][5] )
{
    f(y);
    // DOES match F ; more specialized
    // COULD match I
}
```

§ 5.4. Array Types of Zero Length

An array type of zero length `int[M][N]` where `N==0` is invalid. This can lead to challenges in meta-programming where `N` is computed. A potential work-around enabled by this proposal is to map such a type to an incomplete type.

```
template< size_t M , size_t N >
using T = conditional_t< N != 0 , int[M][N?N:1] , int[M][] > ;
```

§ 5.5. Other Potential Applications

There may be other potential applications for the proposed relaxed incomplete array syntax; for example deduction of multiple dimensions.

Active content removed

§ References

§ Informative References

[N4356]

Carter Edwards. [Relaxed Array Type Declarator](https://wg21.link/n4356). 4 February 2015. URL: <https://wg21.link/n4356>

[N4700]

Richard Smith. [Working Draft, Standard for Programming Language C++ Note.](https://wg21.link/n4700) URL: <https://wg21.link/n4700>

[P0009r0]

H. Carter Edwards, Christian Trott, Juan Alday, Jesse Perla, Mauro Bianco, Robin Maffeo, Ben Sander, Bryce Lelbach. [Polymorphic Multidimensional Array View](https://wg21.link/p0009r0). 23 September 2015. URL: <https://wg21.link/p0009r0>

[P0009r3]

H. Carter Edwards, Bryce Lelbach, Christian Trott, Mauro Bianco, Robin Maffeo, Ben Sander. [Polymorphic Multidimensional Array View](https://wg21.link/p0009r3). 14 October 2016. URL: <https://wg21.link/p0009r3>

[P0331r0]

H. Carter Edwards, Bryce Lelbach, Christian Trott, Mauro Bianco, Robin Maffeo, Ben Sander. [Motivation and Examples for Multidimensional Array](https://wg21.link/p0331r0). 27 May 2016. URL: <https://wg21.link/p0331r0>

[P0332r0]

H. Carter Edwards, Bryce Lelbach, Christian Trott, Mauro Bianco, Robin Maffeo, Ben Sander. [Relaxed Incomplete Multidimensional Array Type Declaration](https://wg21.link/p0332r0). 27 May 2016. URL: <https://wg21.link/p0332r0>

[P0332r1]

H. Carter Edwards, Bryce Lelbach, Christian Trott, Mauro Bianco, Athanasios Iliopoulos, John Michopoulos. [Relaxed Incomplete Multidimensional Array Type Declaration](https://wg21.link/p0332r1). URL: <https://wg21.link/p0332r1>

[P0674r1]

Peter Dimov, Glen Fernandes. [Extending make_shared to Support Arrays](https://wg21.link/p0674r1). URL: <https://wg21.link/p0674r1>